

OBSERVER PATTERN

Observer.java:

```
public interface Observer {  
    void update(String stockName,double price);  
}
```

Stock.java:

```
public interface Stock {  
    void registerObserver(Observer ob);  
    void removeObserver(Observer ob);  
    void notifyObservers();  
}
```

MobileApp.java:

```
public class MobileApp implements Observer {  
    @Override  
    public void update(String stockName,double price) {  
        System.out.println("Mobile App: "+stockName+" updated to Rs."+price);  
    }  
}
```

StockMarket.java:

```
import java.util.ArrayList;  
  
public class StockMarket implements Stock {  
    private ArrayList<Observer> obs=new ArrayList<>();  
    private String stockName;  
    private double price;  
    @Override  
    public void registerObserver(Observer ob) {  
        obs.add(ob);  
    }  
    @Override  
    public void removeObserver(Observer ob) {
```

```

        obs.remove(ob);
    }
    @Override
    public void notifyObservers() {
        for(Observer ob:obs) {
            ob.update(stockName,price);
        }
    }
    public void setStock(String stockName,double price) {
        this.stockName=stockName;
        this.price=price;
        notifyObservers();
    }
}

```

WebApp.java:

```

public class WebApp implements Observer {
    @Override
    public void update(String stockName,double price) {
        System.out.println("Web App: "+stockName+" updated to Rs."+price);
    }
}

```

ObserverPatternTest.java:

```

public class ObserverPatternTest {
    public static void main(String[] args) {
        StockMarket market=new StockMarket();
        Observer mobile=new MobileApp();
        Observer web=new WebApp();
        market.registerObserver(mobile);
        market.registerObserver(web);
        market.setStock("TCS",4200);
        System.out.println();
        market.removeObserver(web);
        market.setStock("Infosys",1800);
    }
}

```

